

最終レポート

大阪大学学生向け教科書取引プラットフォーム 「OU Textbook」の開発

班名: A 班

学籍番号	氏名	電子メールアドレス
25XXXXXX	大阪 太郎	taro.osaka@ecs.osaka-u.ac.jp
25XXXXXX	阪大 花子	hanako.handai@ecs.osaka-u.ac.jp
25XXXXXX	石橋 健二	kenji.ishibashi@ecs.osaka-u.ac.jp

2026 年 7 月 24 日

1 課題 1・2 の概要

1.1 背景と当事者性・切実性

教科書は、授業中には必要で、学期が終わると余りやすい。問題は、この時間差の両側で学生が別々に負担を抱えることである。購入する学生は新品価格や配送費を負担し、手放す学生は梱包・発送の手間を避けるうちに、使わない教科書を本棚に残してしまう。

私たちにも、学期の初めに複数冊をまとめて購入し、授業が終わったあと、まだ使える教科書を譲る相手が見つからないまま保管した経験がある。翌年度の履修者へ渡せばよいとは思っても、誰が同じ授業を取るのか、その人がどのキャンパスにいるのかは分からない。買う側だったときの負担と、売る側になったときの行き詰まりは、同じ教科書の前後に生じていた。

既存のフリマアプリや古本屋でも売買はできる。ただし、大学の授業で使われる特定の版・科目の教科書を探す場面では、次の二点が残る。

1. 配送、梱包、販売手数料などが、少額の中古教科書取引に対して相対的に重い。
2. 書名だけでは授業との対応が分かりにくく、同じ大学・キャンパスの出品へ絞り込めない。

同じキャンパスに通う学生同士なら、配送そのものを省ける。そこで「OU Textbook」では、大阪大学の学生が教科書を検索し、価格と受け渡し日時・場所に合意したうえで、キャンパス内で直接受け渡し流れを一つの Web アプリケーションにまとめた。もっとも、対面取引にすれば問題が消えるわけではない。待ち合わせの不履行と、当事者でない利用者の介入をどう抑えるかが、設計上の焦点となった。

1.2 普遍性と社会性

教科書の購入と処分は、特定の学部や学年だけに生じる問題ではない。豊中・吹田・箕面の各キャンパスで、履修科目が変わるたびに繰り返される。とりわけ新生にとっては、教科書代が入学時の諸費用と同時に発生するため、中古教科書を授業単位で探せる意義は大きい。

再利用には資源面の効果もあるが、本提案の直接的な価値は、まだ使える教科書と、それを必要とする次の履修者を学内で結び付ける点にある。対象を大阪大学の学生へ限定することで利用者数は狭まる一方、キャンパス、授業分野、受け渡し可能性という文脈は濃くなる。この限定が、汎用サービスとの差になる。

1.3 情報技術で解く範囲

送料をなくすだけなら、SNS で相手を募って手渡せばよい。そうしたかった、と思う。しかし、投稿が流れる場所では検索条件、合意内容、取引履歴が分散し、待ち合わせの直前に誰が何へ同意したのかを追いにくい。そこで、次の三つをシステム要件とした。

- **見つける:** 書名・著者名の部分一致検索に、キャンパス、カテゴリ、価格順の条件を組み合わせる。
- **当事者を限定する:** @ecs.osaka-u.ac.jp のメールアドレスを登録条件とし、取引操作は出品者、購入希望者、購入者の権限に応じて制御する。
- **合意を残す:** チャット、価格提示、受け渡し提案、双方の完了確認、相互評価を段階別のデータとして保存する。

信用スコアは最後の要件を補助する。良い評価は +10 点、悪い評価と予定確定後のキャンセルは -10 点、無断不参加の報告は -30 点とし、同じ評価や報告が重複適用されないよう制約を設けた。スコアだけでトラブルを防げるわけではないが、過去の取引結果を次の相手が判断する材料にはできる。

費用をかけずに再現できることも条件とした。Web フレームワークには 3 条項 BSD ライセンスで公開されている Django を採用した [4]。データベースと画像保存は、ローカルでは SQLite とファイルシステムだけで動作する。Supabase もオープンソースの構成要素から成り、Docker を用いたセルフホストが可能である [5]。したがって、外部の有償サービスを契約しなくても、PC 上で認証を含む構成を再現できる。ただし、公開サーバーの維持費まで常にゼロになるわけではない。

1.4 既存手法との比較

メルカリでは商品が売れた際に販売価格の 10% が販売手数料となり、送料は出品時の設定に応じて出品者か購入者が負担する [1, 2]。BOOKOFF は店頭買取に加え、宅配・出張による買取を提供しており、価格は店舗側の査定で決まる [3]。表 1 は、これらの公式情報と、学内 SNS での個別調整を同じ四つの軸で比較したものである。

表 1 既存取引手法と OU Textbook の比較

比較軸	汎用フリマ	一般古本屋	学内 SNS・掲示板	OU Textbook
費用	販売手数料 10%・送料負担あり	買取価格は店舗査定による	原則として当事者間で調整	配送・システム利用料なし
発見性	書名中心。授業との対応は不明	店舗在庫の範囲に限られる	投稿が時系列に流れやすい	キャンパス・カテゴリで絞り込み
受け渡し	主に配送	店舗への持込・来店	個別に調整	日時・場所の提案と双方同意
信頼の手掛かり	運営の評価・補償制度	店舗が取引主体	アカウント情報に依存	学内メール認証・履歴・信用スコア

OU Textbook の強みは、汎用サービスより安全だと言い切れることではない。配送と運営仲介を省く代わりに、授業に即した発見性と受け渡し合意の記録を得る設計である。決済の仲介や本人確認書類による確認は実装していないため、金銭トラブルへの補償は今後も別に考える必要がある。

2 設計・作成したシステム・ソフトウェアの説明

2.1 設計の意図とシステム構成

画面を Django テンプレートで描画し、認証だけを Supabase へ委ねた。ブラウザから認証情報を直接管理する構成ではなく、Django が Supabase Auth の API を呼び出し、返された利用者 ID を Django 標準の User および UserProfile へ同期する。アクセストークンとリフレッシュトークンは Django のセッションに保持する。

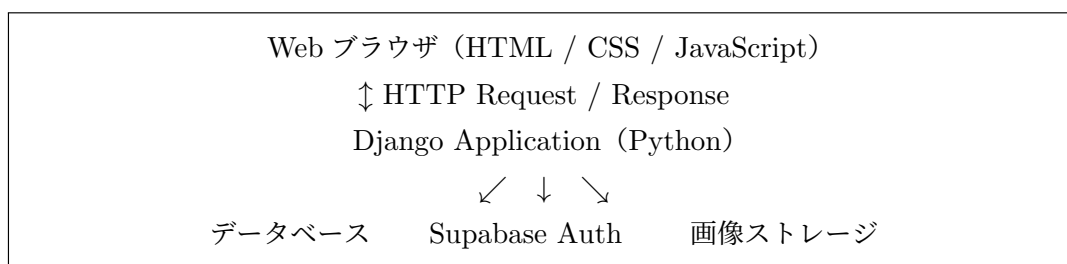


図1 OU Textbook のシステム構成

ローカルでは SQLite を使い、DATABASE_URL が設定された環境では外部データベースへ接続する。画像は、所定の環境変数がある場合は Supabase Storage へ、ない場合はローカルファイルシステムへ保存する。

認証サービスを分離しても、取引データの整合性は Django 側に残る。価格提示の承諾、受け渡しの確定、完了確認、評価の適用は、状態と操作者の両方を検査してから更新する。第三者が URL を推測しても取引を確定できないことが必要だった。

2.2 データモデルと取引状態

実装した中心的なモデルは、次の八つである。

UserProfile 表示名、学部・学科、学年、Supabase 利用者 ID、信用スコアを保持する。スコアに応じて「要注意」「レギュラー」「トラスト」「エキスパート」の表示を切り替える。

Book 書名、著者、価格、カテゴリ、キャンパス、状態、画像、出品者と購入者を保持する。状態は available、in_progress、sold、withdrawn の四つである。

Favorite / Message お気に入り利用者と書籍の組を一意に保存する。メッセージは書籍、送信者、受信者、本文、送信時刻を保存し、書籍ごとの相談履歴を構成する。

TradeOffer 出品者から特定の購入希望者への価格提示を保存する。一冊に対して成立済みの提示が複数残らないよう、条件付き一意制約を置いた。

HandoffProposal 成立した価格提示に対する受け渡し日時・場所と、売り手・買い手双方の完了確認時刻を保存する。

Evaluation / CancellationLog 相互評価とキャンセル・無断不参加の報告を保存する。一意制約と適用済みフラグにより、信用スコアの二重加算・減算を防ぐ。

取引は、相談を始めただけでは成立しない。出品者が価格を提示し、対象の購入希望者が承諾した時点で、書籍は `available` から `in_progress` へ移る。受け渡し予定時刻を過ぎた後、双方が完了を確認し、双方の評価がそろそろ `sold` になる。途中でキャンセルされた場合は購入者との紐付けを外し、再び `available` へ戻す。

2.3 メンバー間の作業分担方針と実際の作業分担結果

画面、取引ロジック、外部サービス統合は相互に影響するため、担当を分けつつ、境界部分を共同で確認する方針を採った。

- 大阪 太郎 (学籍番号: 25XXXXXX) : バックエンド開発統括。Django のモデル設計、書籍登録、取引状態制御、信用スコア処理を担当した。
- 阪大 花子 (学籍番号: 25XXXXXX) : フロントエンド UI/UX 開発。書籍一覧、詳細、チャット、マイページの画面設計とレスポンス対応を担当した。
- 石橋 健二 (学籍番号: 25XXXXXX) : インフラ・認証統合。Supabase Auth と画像ストレージの統合、環境設定、権限制御と動作テストを担当した。

実際の作業でも三つの担当領域は維持した。ただし、画面から取引状態を変更する機能については、フロントエンドだけ、あるいはバックエンドだけでは完結しない。価格提示、受け渡し確認、評価の各画面は、画面担当と取引ロジック担当が共同で動作を確認し、認証や保存先に関わる不具合はインフラ担当を交えて修正した。

2.4 動作環境とローカル構築手順

依存バージョンは `requirements.txt` で固定または範囲指定している。主な構成は Django 6.0.6、SQLite (ローカル)、Supabase Auth である。本番用設定の `render.yaml` では Python 3.14.3 を指定し、`DATABASE_URL` による外部データベース接続、`WhiteNoise` による静的ファイル配信、`Supabase Storage` による画像保存に対応した。

Windows PowerShell でソースコードを取得済みの場合、次の手順で起動できる。

```
cd C:\path\to\pbl

python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r trading_text\requirements.txt

$env:SUPABASE_URL="https://xxxx.supabase.co"
$env:SUPABASE_ANON_KEY="your-supabase-anon-key"

cd trading_text
python manage.py migrate
python manage.py check
python manage.py runserver
```

画像を Supabase Storage へ保存する場合は、これらに加えて `SUPABASE_STORAGE_BUCKET` と `SUPABASE_SECRET_KEY` を設定する。秘密鍵はソースコードへ書かず、環境変数として渡す。

2.5 動作結果

開発時には、画面操作による主要シナリオの確認に加え、`main/tests.py` へ 72 件のテストメソッドを作成した。確認対象と判定条件を表 2 に示す。

表 2 主要な確認対象と判定条件

対象	操作・条件	確認内容
登録・ログイン	ECS 以外／ECS のメールを入力	前者を拒否し、後者は Supabase 利用者 ID とローカル利用者を同期する
検索・出品	書名・著者、キャンパス、カテゴリ、価格順を指定	条件に合う <code>available</code> の書籍だけを表示する
価格合意	出品者が提示し、対象の購入希望者が承諾	提示価格を保存し、書籍を <code>in_progress</code> へ移す
受け渡し	日時・場所へ合意し、予定時刻後に双方が完了確認	一方だけでは完了にせず、第三者の操作も受け付けない
評価・キャンセル	双方評価、重複評価、キャンセル、無断不参加	所定の点数を一度だけ反映し、キャンセル時は出品を再開する
画像	5MB 超または未対応シグネチャのファイルを送信	バリデーションエラーとし、保存しない

とくに取引状態は、画面上の表示だけでなく、当事者以外による価格承諾、受け渡し確定、完了確認を拒否できるかまで確認した。正常系だけなら一冊の売買は進む。状態が分岐する拒否、再提案、キャンセル、重複送信で整合性を失わないことのほうが、実装上は重要だった。

2.6 最も苦勞した点と次に苦勞した点

2.6.1 最も苦勞した点: Supabase Auth と Django 標準認証の統合

Supabase Auth の利用者と Django の利用者は、同じ人物を表していても別の識別子と状態を持つ。認証成功後にメールアドレスだけで結び付ければ簡単に見えるが、表示名の初期化、既存利用者との同期、Supabase 利用者 ID の保存、Django セッションの開始までを一貫させなければならない。

そこで、Supabase の認証 API を呼ぶ処理を `supabase_auth.py` へ分離し、ビュー側の `sync_supabase_user` でローカル利用者とプロフィールを同期した。認証サービスのエラーをフォームへ返しつつ、成功時だけ Django のログイン状態を作る境界の調整に最も時間を要した。

2.6.2 次に苦労した点: アップロード画像の検査と保存先の切替

拡張子だけを検査しても、ファイルの実体は判別できない。実装では 5MB の上限を設け、先頭 16 バイトを JPEG、PNG、GIF、WebP の候補シングネチャと照合してから保存するようにした。また、ローカルと本番で保存先が変わっても Book.image の扱いを変えずに済むよう、Django の Storage インターフェースを用いた。

ただし、先頭部分の照合は完全な画像デコードではない。特に RIFF 形式は WebP 以外にも使われるため、現在の検査だけで悪意あるファイルを排除できるとは言い切れない。この限界は未解決である。

2.7 未解決の部分と今後の課題

実装を終えても、対面取引の不確実さは残った。限られた期間で、検索、価格合意、受け渡し、完了確認、評価という取引の中心経路と、第三者の操作を拒否する権限制御を優先したためである。外部サービスとの継続的な通信が必要な機能と、運用者の判断基準が必要な機能は、短いローカル試験だけでは安全性を確かめられず、実装範囲から外した。

優先度の高い順に、次の改善が必要である。

1. **認証セッションの更新と失効処理:** 現在は Supabase から得たアクセストークンとリフレッシュトークンをセッションへ保存するが、期限切れ時の自動更新と、Supabase 側で失効した利用者をローカルセッションへ反映する処理が不足している。原因は、登録・ログインの成立を先に検証し、時間経過や外部での失効を含む長時間試験まで実施できなかったことである。
2. **画像の完全な検証:** Pillow 等で画像を実際にデコードし、形式、画素数、破損の有無を検査したうえで再エンコードする必要がある。現在の検査は追加ライブラリを用いず先頭バイトだけで判定する方針を採ったため、RIFF コンテナの内部形式まで識別できなかった。
3. **チャット通知の即時化:** 現在は画面の再読み込みで新着メッセージを取得する。Django Channels 等によるリアルタイム更新、またはメール通知が必要である。通常の HTTP による取引機能を先に完成させた結果、WebSocket 接続の維持と切断時の再接続を検証する期間を確保できなかった。
4. **受け渡し場所の選択支援:** 現在は場所を自由記述する。学内の明るく人通りのある候補地点を選択式で示し、必要に応じて地図上で確認できる形が望ましい。候補地点の選定には各キャンパスの現地確認が必要であり、API を接続するだけでは安全な場所を保証できないため、自由記述にとどめた。
5. **紛争対応と運用設計:** 学内メール認証と信用スコアは本人確認や補償の代替ではない。虚偽の無断不参加報告、金銭トラブル、不適切な出品に対する異議申立てと管理者対応を定める必要がある。これはプログラムだけでは決められず、証拠の扱い、管理者の権限、個人情報情報の保存期間について合意できる運用主体が必要になる。

3 結論

余っている教科書と、次の学期にそれを必要とする学生は、同じキャンパスにいる。それでも取引が成立しにくかったのは、相手を見つける情報と、価格・日時・場所への合意が別々の手段に散らばっていたからである。

OU Textbook は、学内メール認証、教科書検索、チャット、価格提示、受け渡し合意、完了確認、評価を一つの状態遷移として実装した。これにより、配送を使わない教科書取引を、単なる呼びかけから記録可能な手続きへ変えられた。一方で、認証失効、画像の完全検査、紛争対応は残っている。学期が変わるたびに安心して使える仕組みになるかどうかは、この三点を運用まで含めて詰められるかにかかっている。

参考文献

- [1] メルカリ「価格の決め方」, <https://help.jp.mercari.com/guide/articles/64/> (2026 年 7 月 24 日閲覧).
- [2] メルカリ「送料はどちらが負担するのでしょうか?」, <https://help.jp.mercari.com/guide/articles/107/> (2026 年 7 月 24 日閲覧).
- [3] BOOKOFF「売りたい」, <https://www.bookoff.co.jp/sell/> (2026 年 7 月 24 日閲覧).
- [4] Django Software Foundation, “FAQ: General — How is Django licensed?”, <https://docs.djangoproject.com/en/6.0/faq/general/> (2026 年 7 月 24 日閲覧).
- [5] Supabase, “Self-Hosting with Docker”, <https://supabase.com/docs/guides/self-hosting/docker> (2026 年 7 月 24 日閲覧).